

All Stack Functions in C

Complete array-based implementation

Unit I • Lecture 8

`init • isEmpty • isFull • size • push • pop • peek • clear • display`

Define the stack before writing its functions

HEADERS + STACK TYPE

```
#include <stdio.h>
#include <stdbool.h>
#include <stddef.h>

#define STACK_CAPACITY 5

typedef struct {
    int items[STACK_CAPACITY];
    int top;
} Stack;
```

Representation

items stores values.
top stores the current
top index.

Valid top values:
-1 through
STACK_CAPACITY - 1

Declare the complete stack interface

FUNCTION PROTOTYPES

```
void    stackInit(Stack *s);
bool    stackIsEmpty(const Stack *s);
bool    stackIsFull(const Stack *s);
size_t  stackSize(const Stack *s);

bool    stackPush(Stack *s, int value);
bool    stackPop(Stack *s, int *outValue);
bool    stackPeek(const Stack *s, int *outValue);

void    stackClear(Stack *s);
void    stackDisplay(const Stack *s);
```

stackInit creates an empty stack

INITIALIZE

```
void stackInit(Stack *s) {  
    s->top = -1;  
}
```

Call

```
Stack s;  
stackInit(&s);
```

**After initialization:
stackSize(&s) == 0**

SAMPLE CALL

```
stackInit(&s);
```

stackIsEmpty tests the empty sentinel

EMPTY CHECK

```
bool stackIsEmpty(const Stack *s) {  
    return s->top == -1;  
}
```

Returns

true → no active element
false → at least one element

SAMPLE CALL

```
if (stackIsEmpty(&s))  
    printf("Stack is empty\n");
```

**const prevents this
function
from changing the stack.**

stackIsFull protects the array boundary

FULL CHECK

```
bool stackIsFull(const Stack *s) {  
    return s->top == STACK_CAPACITY - 1;  
}
```

Returns

true → no free slot
false → push may proceed

SAMPLE CALL

```
if (stackIsFull(&s))  
    printf("Stack overflow\n");
```

For capacity 5, the last valid array index is 4.

stackSize converts the top index into a count

SIZE

```
size_t stackSize(const Stack *s) {  
    return (size_t)(s->top + 1);  
}
```

Examples

top = -1 → size = 0
top = 0 → size = 1
top = 3 → size = 4

Size is always top + 1.

SAMPLE CALL

```
printf("Size: %zu\n", stackSize(&s));
```

stackPush inserts only when space is available

PUSH

```
bool stackPush(Stack *s, int value) {  
    if (stackIsFull(s))  
        return false;  
  
    s->items[++s->top] = value;  
    return true;  
}
```

SAMPLE CALL

```
stackPush(&s, 25);
```

Success

Increment top, store value, and return true.

Overflow

Return false. The stack remains unchanged.

stackPop returns the top value and removes it

POP

```
bool stackPop(Stack *s, int *outValue) {  
    if (stackIsEmpty(s))  
        return false;  
  
    *outValue = s->items[s->top--];  
    return true;  
}
```

SAMPLE CALL

```
int x;  
if (stackPop(&s, &x)) printf("%d\n", x);
```

Success

Copy the top value, decrement top, and return true.

Underflow

Return false. outValue is not changed.

stackPeek reads the top without removing it

PEEK

```
bool stackPeek(const Stack *s, int *outValue)
{
    if (stackIsEmpty(s))
        return false;

    *outValue = s->items[s->top];
    return true;
}
```

SAMPLE CALL

```
int x;
if (stackPeek(&s, &x)) printf("Top: %d\n",
x);
```

Success

Copy items[top] and return true.

top does not change.
The stack remains unchanged.

stackClear resets the logical contents

CLEAR

```
void stackClear(Stack *s) {  
    s->top = -1;  
}
```

Call

```
stackClear(&s);
```

**Old bytes may remain, but
no element is active.**

SAMPLE CALL

```
stackClear(&s);
```

stackDisplay prints from top to bottom

DISPLAY

```
void stackDisplay(const Stack *s) {  
    if (stackIsEmpty(s)) {  
        printf("[empty]\n");  
        return;  
    }  
  
    printf("TOP -> ");  
    for (int i = s->top; i >= 0; --i)  
        printf("%d ", s->items[i]);  
    printf("\n");  
}
```

Output order

The top value is printed first, followed by lower values.

Example:
TOP -> 30 20 10

Complete source — declarations and queries

SOURCE 1 OF 3

```
#include <stdio.h>
#include <stdbool.h>
#include <stddef.h>
#define STACK_CAPACITY 5

typedef struct {
    int items[STACK_CAPACITY];
    int top;
} Stack;

void stackInit(Stack *s) {
    s->top = -1;
}
```

QUERY FUNCTIONS

```
bool stackIsEmpty(const Stack *s)
{
    return s->top == -1;
}

bool stackIsFull(const Stack *s) {
    return s->top ==
STACK_CAPACITY - 1;
}

size_t stackSize(const Stack *s) {
    return (size_t)(s->top + 1);
}
```

Complete source — push, pop, and peek

PUSH

```
bool stackPush(
    Stack *s, int
    value)
{
    if
    (stackIsFull(s))
        return false;
    ++s->top;
    s->items[s->top]
        = value;
    return true;
}
```

POP

```
bool stackPop(
    Stack *s, int
    *out)
{
    if
    (stackIsEmpty(s))
        return false;
    *out = s-
    >items[s->top];
    --s->top;
    return true;
}
```

PEEK

```
bool stackPeek(
    const Stack *s,
    int *out)
{
    if
    (stackIsEmpty(s))
        return false;
    *out = s-
    >items[s->top];
    return true;
}
```

Complete source — clear and display

SOURCE 3 OF 3

```
void stackClear(Stack *s) {
    s->top = -1;
}

void stackDisplay(const Stack *s) {
    if (stackIsEmpty(s)) {
        printf("[empty]\n");
        return;
    }
    printf("TOP -> ");
    for (int i = s->top; i >= 0; --i)
        printf("%d ", s->items[i]);
    printf("\n");
}
```

Test every stack function in one program

MAIN — PART 1

```
int main(void) {  
    Stack s;  
    int x;  
    stackInit(&s);  
    printf("Empty: %s\n",  
        stackIsEmpty(&s) ? "yes" : "no");  
    stackPush(&s, 10);  
    stackPush(&s, 20);  
    stackPush(&s, 30);  
    stackDisplay(&s);
```

MAIN — PART 2

```
        if (stackPeek(&s, &x))  
            printf("Peek: %d\n", x);  
        if (stackPop(&s, &x))  
            printf("Pop: %d\n", x);  
        printf("Size: %zu\n",  
            stackSize(&s));  
        stackClear(&s);  
        printf("Cleared\n");  
        return 0;  
}
```

Empty: yes • TOP -> 30 20 10 • Peek: 30
Pop: 30 • Size: 2 • Cleared